

South African Bank Notes Recognition

COMP702 - Image Processing and Computer Vision Project Report

Marry.K.221011344, Shah.M.221021204, Ntanzi.S.221039353, Ramjan.Y.221004686

UNIVERSITY OF KWAZULU-NATAL SCHOOL OF MATHEMATICS, STATISTICS AND COMPUTER SCIENCE

June, 2024

Abstract

Bank note recognition involves using various image processing and computer vision techniques to capture and classify images of paper-based currency. This process is crucial for counterfeit detection, automated cash handling, improved efficiency in banking operations, and various other security measures. This project aims to explore different techniques to accurately classify both old and new South African banknotes, specifically in denominations of R10, R20, R50, R100, and R200.

1. Introduction

Humans possess an extraordinary ability for recognizing patterns and abstract features, enabling us to effortlessly differentiate between various banknote denominations in business transactions. This cognitive ability allows us to determine the value of banknotes even when only part of the note is visible. Our brains efficiently process visual information, swiftly categorizing and identifying different denominations. Artificial neural networks (ANNs) aim to replicate the human brain's functioning as they are inspired by human cognition. ANNs strive to emulate our pattern recognition and data comprehension abilities. Through the study and advancement of these systems, we can deepen our understanding of human cognition and potentially enhance the capabilities of artificial intelligence.

In this paper, we will use various image processing techniques that include image acquisition, image processing and enhancement, image segmentation, feature extraction, image classification, and compare these results. The bank note recognition system should compute a similarity measure between notes that are invariant to sides, scale and rotations, since a bank note has two faces, and the orientation of the note can be any angle.

2. Related Work

2.1. Support Vector Machine (SVM)

In this section, we will introduce the possible models we plan to implement. SVMs are powerful supervised machine learning algorithms that is used for both classification and regression, but it is most effective for classification tasks. According to [2], the main goal of the SVM algorithm is to find the optimal hyperplane in an N-dimensional space to separate data points of different classes in the feature space. This is done with the aim of maximizing the margin between the closest points of different classes, making

SVM adaptable and efficient for various applications. According to [5], it can handle high-dimensional data and nonlinear relationships, and is designed to effectively handle two-class classification problems. The fundamental principle of SVM is to build a suitable separating hyperplane as the decision surface in order to maximize the difference in distance between two patterns.

[4] Sufri et al, states that utilizing a SVM with a linear kernel is highly effective for bank note recognition due to several crucial reasons. First, SVMs excel in high-dimensional spaces, effectively separating bank note classes by handling features such as textures, shapes, edges, and color moments[1]. Second, the distinct features of bank notes make a linear kernel ideal for separating classes with a hyperplane[1]. Third, SVMs are designed to maximize the margin between well-defined classes, making them perfectly suited for distinguishing various bank note denominations. Furthermore, linear SVMs are computationally efficient and easier to implement, making them practical for processing large datasets[1]. Lastly, linear SVMs with a large margin generalize effectively to new data, ensuring accurate classification of unseen bank notes.

2.2. Feed-Foward Neural Network (FNN)

The potential of utilizing a FNN for banknote recognition is explored in this section. [6] gives reasons and explains the characteristics that we can use to show why FNNs are well-suited for this task due to their ability to learn complex patterns and relationships in high-dimensional data. They excel at capturing intricate patterns and features present in banknotes, can handle nonlinear relationships, offer scalability and adaptability to diverse datasets, and advancements in deep learning frameworks and hardware accelerators have made training FNNs feasible even for large-scale datasets. Overall, by leveraging the capabilities of FNNs, the aim is to build a robust banknote recognition system capable of accurately identifying and classifying different denominations of banknotes based on their distinct features.

2.3. Random Forest Classifier

[3] The Random Forest algorithm is a powerful tree-learning technique in machine learning. It works by creating several decision trees during the training phase. Each tree is constructed using a random subset of the dataset to measure a random subset of features in each partition. This randomness introduces variability among individual trees, reducing the risk of overfitting and improving overall prediction performance. In prediction, the algorithm aggregates the results of all trees, either by voting (for classification tasks) or by averaging (for regression tasks). This collaborative decision-making process, supported by multiple trees with their insights, provides stable and precise results. Random forests are widely used for classification and regression functions. They are known for their ability to handle complex data, reduce overfitting, and provide reliable forecasts in different environments.

3. Methods and Techniques

3.1. Bank Note Image Preprocessing and Enhancement

3.1.1. Dataset

The dataset for this investigation utilizes South African banknotes, the official currency of South Africa. The primary goal of this study is to explore the cognitive framework involved in the recognition of banknotes. Therefore, we will be including all denominations of the banknotes (R10, R20, R50, R100, and R200).

Table 1: Numbers of bank note images in the dataset.¹

<i>Type of Bank Note</i> ²	<i>Front</i>	<i>Back</i>
R10	5	5
Recent R10	2	2
R20	3	3
Recent R20	1	1
R50	4	4
Recent R50	2	2
R100	4	4
Recent R100	2	2
R200	3	3
Recent R200	1	1

The dataset of bank note images were converted to a `.json` file that contains the descriptions, Base64 encoded images (back and front), and values of South African bank notes. Code is implemented to decode the images, extract the required features from both the back and front images, as well as compile these features into a `DataFrame` along with their corresponding labels. This processed data will then be used for further analysis and machine learning tasks, such as classifying different bank notes. Visualisation of the characteristics of the `.json` file can be seen in appendix A.8.

Sample images showing the denominations are shown in fig. 1. From fig. 1, the South African banknotes used are shown with both front and back sides of each denomination. Overall, the dataset has 54 images with 5 denominations of 3 versions each (Current version(2024), young Nelson Mandela at the back, without young Nelson Mandela at the back(has the animal)).

3.1.2. Preprocessing and Enhancement

Since humans are able to recognize banknotes of familiar a currency almost effortlessly after a period of using such banknotes. The original colour images used within this work are processed such that they are suitable to be fed as neural network inputs. Hence, the sequence below is used to obtain the final neural networks inputs:

1. **Gray-Scale Conversion.** Converting a colour image to gray-scale simplifies the image data by reducing it to one channel instead of three (RGB). A sample can be shown in fig. 2 of the R10 bank note. This is useful because many image processing techniques, such as edge detection and thresholding, are more efficient and effective on gray-scale images. Gray-scale images retain the essential structural information while reducing computational complexity.
2. **Contrast Enhancement (Histogram Equalization)** Enhancing the contrast of bank note images helps in making the features more distinguishable. A sample can be shown in fig. 3 of the R10 bank note. Bank notes often contain intricate designs, watermarks, and fine details that are crucial for accurate recognition. Histogram equalization enhances the visibility of these features, making

¹The types of bank notes also include the rotated images for each denomination.

²There were two version of previous generation bank notes: 1. With young Nelson Mandela, 2. Without young Nelson Mandela, both of which are at the back of the notes.

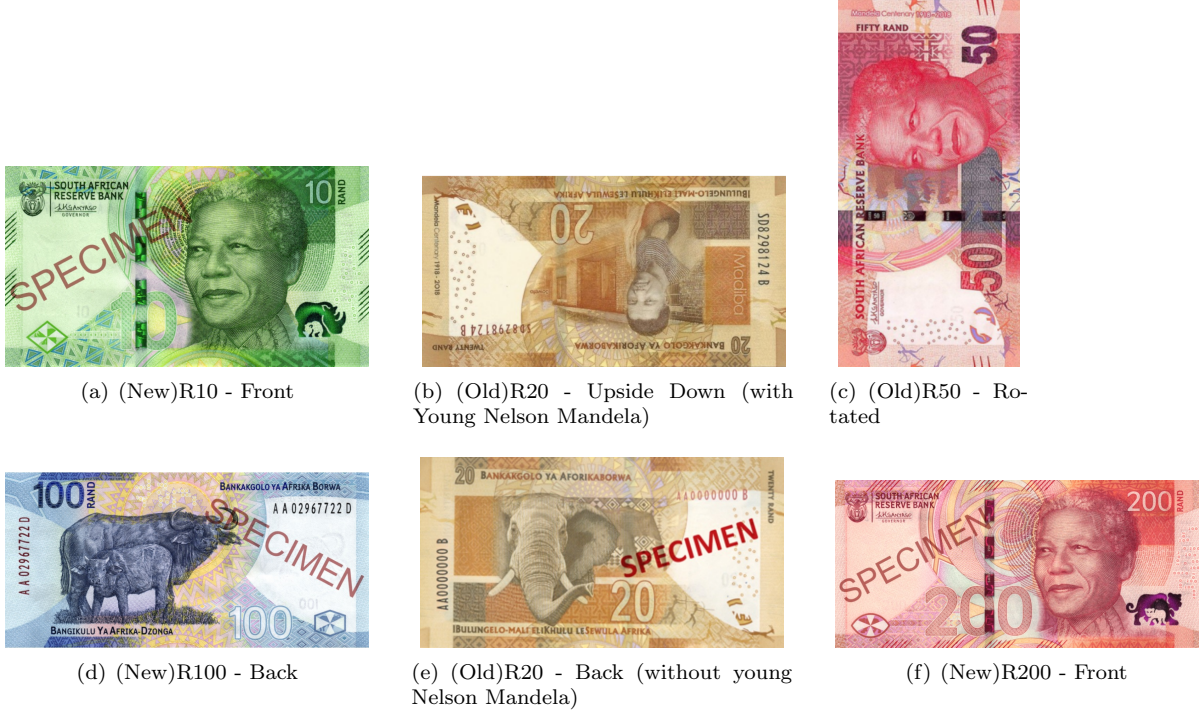


Figure 1: A portion of the dataset. Front and back SA bank notes, both old and new(current) versions.

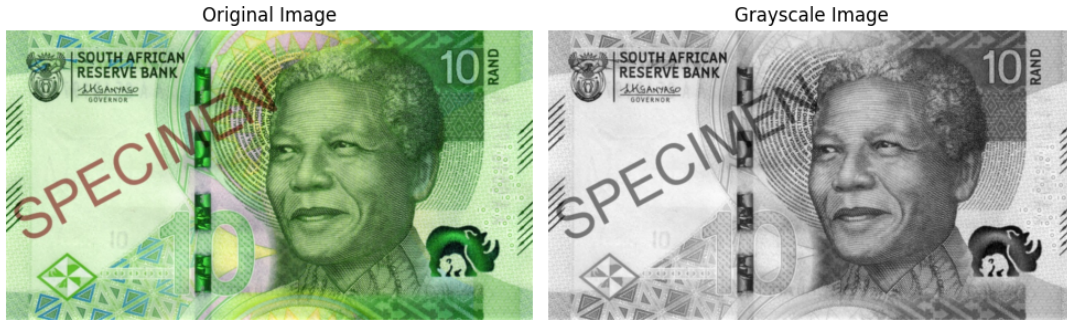


Figure 2: Example of Gray-Scale Conversion on the R10 bank note.

it easier for subsequent image processing steps to detect and analyze them.

3. **Noise Reduction** Bank notes, like any other physical object, can be subject to various types of noise and imperfections during image capture. This noise can arise from the camera sensor, environmental conditions, or the surface condition of the notes themselves. A sample can be shown in fig. 4 of the R10 bank note. Applying noise reduction techniques helps to smooth out imperfections, resulting in cleaner images which is crucial to improve feature extraction and classification accuracy.

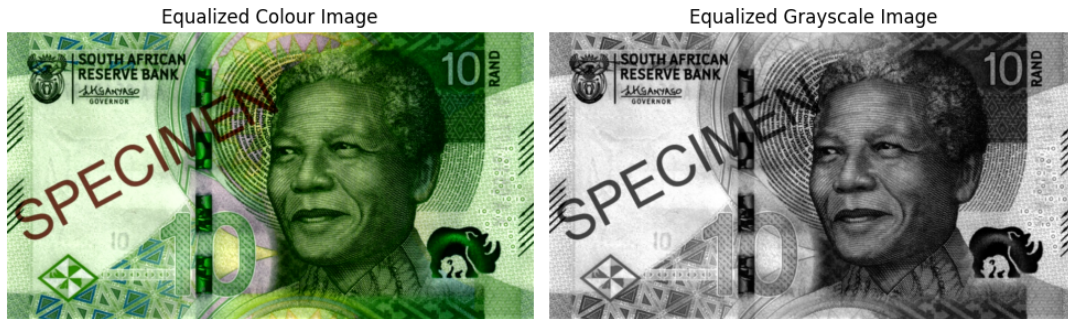


Figure 3: Example of Histogram Equalization on the both the colour and gray-scale R10 bank note.

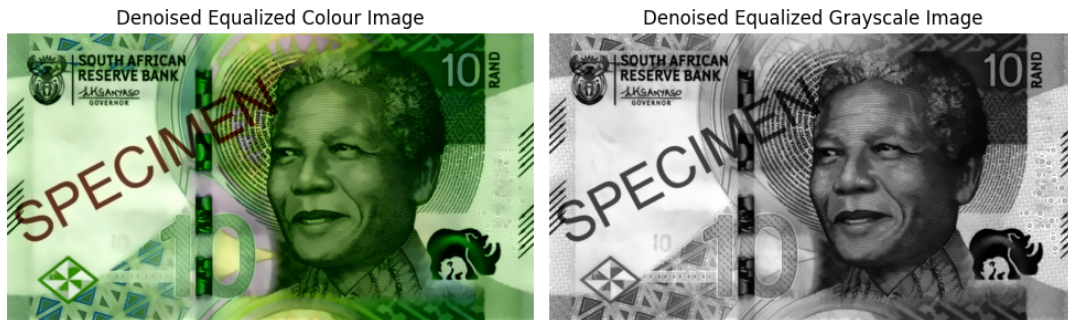


Figure 4: Example of Noise Reduction on the both the colour and gray-scale R10 bank note.

4. **Image Thresholding** Thresholding converts gray-scale images into binary images by setting pixel values above a certain threshold to white (255) and below to black (0). A sample can be shown in fig. 5 of the R10 bank note. This is particularly useful for segmenting the image into regions of interest. For bank notes, this can help in isolating key features such as text, serial numbers, or specific design elements from the background.
5. **Edge Detection** Edge detection identifies the boundaries of objects within an image. For bank notes, edge detection, specifically *Canny Edge Detection*³, can highlight the outlines of various features such as portraits, signatures, and borders, which can be seen in fig. 6. These edges are crucial for feature extraction algorithms, which rely on the precise location of these boundaries to identify and classify different denominations and designs accurately.

³Canny Edge Detection is a robust and accurate method for detecting edges in images, outperforming simpler techniques like Sobel and Prewitt. However, it is computationally intensive and sensitive to parameter settings.



Figure 5: Example of Thresholding on the denoised gray-scale R10 bank note.

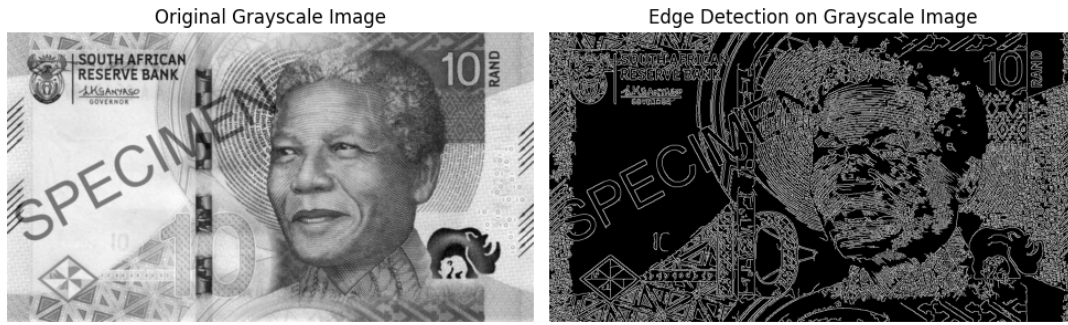


Figure 6: Example of Edge Detection on the denoised gray-scale R10 bank note.

3.2. Bank Note Segmentation

Each segmentation technique has its own strengths and is suitable for different aspects of bank note recognition. Thresholding techniques are simple and effective for clear foreground-background separation, while edge-based and region-based methods are useful for detecting intricate patterns and regions. Clustering and morphological methods help refine the segmentation, and machine learning-based techniques offer high accuracy for complex tasks.

3.3. Bank Note Features Extraction

Feature extraction is essential for reducing data complexity, improving interpretability, and enhancing the accuracy and robustness of the recognition system.

The decision was to use the Haralick features⁴, Hu moments⁵, edge features⁶, and color moments⁷ from images to create a comprehensive feature set for banknote recognition. These features capture different

⁴Captures texture properties.

⁵Describes shape invariants.

⁶Quantifies edge density and patterns

⁷Represents the colour distribution in the image.

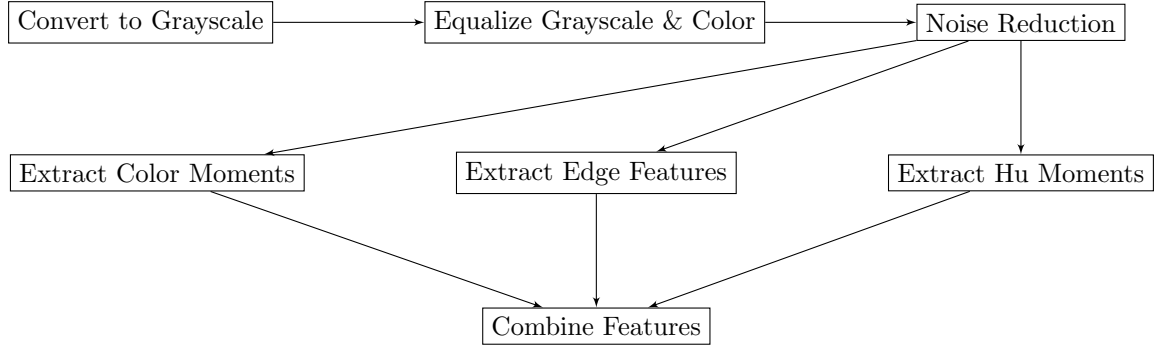


Figure 7: Pipeline for Feature Extraction

aspects of the banknotes - texture, shape, edges, and color - providing a rich and robust representation for accurate classification.

<i>BankNote</i>	<i>Side</i>	<i>NumberofFeatures</i> ⁸	<i>NumberofFeatures</i> ⁹
R10	Front	17	22
R10	Back	17	22
R20	Front	17	22
R20	Back	17	22
R50	Front	17	22
R50	Back	17	22
R100	Front	17	22
R100	Back	17	22
R200	Front	17	22
R200	Back	17	22

Table 2: Bank Note Features.

Figure 7 shows a pipeline of how the main function coordinates the extraction of various features, including Hu moments, edge features, and color moments from processed images, and then concatenates these feature vectors to return a combined feature vector for the input image.

The following techniques were used in the classification aspect:

- **Hu Moments**

- Extracts Hu moments from a grayscale image. The Hu moments are a set of seven numerical values that are invariant to image transformations like translation, rotation, and scaling. It first computes the moments of the image using OpenCV's `cv2.moments()` function, then calculates the Hu moments using `cv2.HuMoments()`, and finally flattens the result to a list.

⁸This is a combination of Hu moments, edge features, and colour moments.

⁹Only included 5 of the 13 Haralick features of the bank notes, but opted not to use due to poor performances.

- **Edge Features**

- Extracts edge features from an edge-detected image. It calculates the edge density by summing up the edge pixel values and dividing by the total number of pixels in the image.

- **Colour Moments**

- Extracts color moments from a color image. It splits the image into its Red, Green, and Blue channels and computes the mean, standard deviation, and skewness for each channel. Skewness measures the asymmetry of the distribution of pixel intensities in each color channel. Calculating the skewness of a given channel removes any NaN (not a number) values from the channel and calculates the skewness using the `skew()` function from NumPy’s `scipy.stats` module, and returns the result.

- **Combine Features**

- The extraction of various features from an input image. It first converts the image to grayscale, equalizes both the grayscale and color images to enhance contrast, reduces noise in both grayscale and color images, then extracts Hu moments, edge features, and color moments from the processed images. Finally, it concatenates these feature vectors and returns the combined feature vector for the input image. Table 3 shows the number of features for each step of the feature extraction process, while appendix A.5 shows the features extracted from the current (2024) South African R10 bank note.

Table 3: Features for each South African Bank Note.¹⁰

<i>Type of Bank Note</i> ¹¹	<i>Hu Moment</i>	<i>Edge Feature</i>	<i>Colour Moment</i>	<i>Combined Features</i>
R10	7	1	9	17
Recent R10	7	1	9	17
R20	7	1	9	17
Recent R20	7	1	9	17
R50	7	1	9	17
Recent R50	7	1	9	17
R100	7	1	9	17
Recent R100	7	1	9	17
R200	7	1	9	17
Recent R200	7	1	9	17

3.4. Bank Note Classification

The primary purpose of classification in bank note recognition is to automate the process of identifying and distinguishing between different bank notes based on their characteristics. This automation leads to improved efficiency, security, and accuracy in handling bank notes. By making use of figure 7, we

¹⁰Features are the same for both front and back sides of the bank notes.

¹¹There were two version of previous generation bank notes: 1. With young Nelson Mandela, 2. Without young Nelson Mandela, both of which are at the back of the notes.

can implement the classification of our proposed models in section 2 by defining several functions for extracting different types of features from an image. It includes functions for extracting Hu moments from a grayscale image, edge features from an edge-detected image, and color moments from a color image. Additionally, there is a function to calculate skewness for a given channel, these functions be seen in appendix A.9.

3.4.1. SVM Classification

Implementation and evaluating the SVM regarding classification involves a series of steps, which starts by reading in another `.json` file for the testing dataset¹². We initialize empty lists to store the descriptions, images, and values of the bank notes. Then, we iterate through the JSON data and populate these lists with the respective information. After collecting the data, we create a pandas DataFrame `df_test` with columns for description, image, and value.¹³

For each row in the DataFrame `df_test`, we decode the base64-encoded image, extract features from the image using the `extract_features` function, and append these features to the `features_test`. We also collect the corresponding labels (values) into a separate list called `labels_test`. Next was to convert `features_test` and `labels_test` into a pandas DataFrame `features_test_df`. The features are stored in the DataFrame, and the labels are stored in a column named `label`.¹⁴

Standardizing the feature values uses `StandardScaler` to ensure that we get a mean of 0 and a standard deviation of 1. This step is crucial for SVM models as they are sensitive to the scale of input features. The process involving the training of the SVM model with a linear kernel used the standardized training data. The model is then used to make predictions on the standardized test data¹⁵.

3.4.2. FNN Classification

This process involves building and training a feed-forward neural network (FNN) model using the `Keras` library with 50 epochs. The training and testing labels are encoded and converted into one-hot encoded categorical arrays using `LabelEncoder` and `to_categorical` respectively. The FNN model consists of three hidden layers with ReLU activation functions and dropout layers to prevent overfitting. The model is compiled with categorical cross-entropy as the loss function and the Adam optimizer, then trained using the fit method with a validation split of 20% and a batch size of 8.

Next was to define a FNN model using the `Sequential` API of Keras. It consists of three hidden layers ('Dense') with ReLU activation functions. Dropout layers with a dropout rate of 0.5 are added after each hidden layer to prevent overfitting of the model.

3.4.3. Random Forest Classifier Classification

The steps used are the same as sections 3.4.1 and 3.4.2, with the only difference is defining a `RandomForestClassifier`. The `RandomForestClassifier` is initialized with 100 trees and trained on the training

¹²This testing dataset will also be used in sections 3.4.2 and 3.4.3

¹³This was also used in sections 3.4.2 and 3.4.3.

¹⁴This was also used in sections 3.4.2 and 3.4.3.

¹⁵The accuracy of the model here an in sections 3.4.2 and 3.4.3 are calculated by comparing the predicted labels with the actual labels, and a detailed classification report is printed, including precision, recall, F1-score, and support for each class.

dataset. The accuracy of the trained model is evaluated using the test dataset. We decided to use a cross-validation of 5 folds, which is used on the training data to obtain a good estimate of the model's performance.

4. Results and Discussion

The results obtained in table 4 shows how each model performs when seeing an unknown test set of size 16. The metrics used for evaluation are accuracy, precision, recall, and f1-score.

Machine Learning Classification Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
Support Vector Machine (SVM-Linear Kernel)	87.50	82.35	87.50	84.89
Feed-Forward Neural Network	93.75	100.00	93.75	96.91
Random Forest Classifier	93.75	100.00	93.33	96.37

Table 4: Machine Learning Classification Model Performance

By measuring the accuracy of the model, we can measure the proportion of correctly classified predictions out of the total predictions and from table 4, we can see that both the Feed-Foward Neural Network and Random Forest Classifier have the joint highest regarding accuracy with a value of 93.75%.

When measuring the precision, we measure the proportion of true positive predictions out of all actual positive predictions made by the model. Based on table 4, we can see both the Feed-Foward Neural Network and Random Forest Classifier have the joint highest regarding precision with a value of 100% and the Support Vector Machine has the lowest with 82.35%. It is given by the formula:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1)$$

where:

- TP = True Positives
- FP = False Positives

Measuring the recall (sensitivity) would allow us to measure the proportion of true positive predictions out of all actual positives (i.e. how many relevant items are selected) and is important when the cost of false negatives is high. Based on table 4, we can see that the Feed-Foward Neural Network has the highest recall with a value of 93.75% and the Support Vector Machine has the lowest with 87.50%. It is given by the formula:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

where:

- TP = True Positives
- FN = False Negatives

Calculating the f1-score combines precision and recall into a single metric by calculating their harmonic mean. It balances the two metrics, providing a single measure that accounts for both false positives and false negatives. Based on table 4, we can see that the Feed-Foward Neural Network has the highest f1-score with a value of 96.91% and the Support Vector Machine has the lowest with 84.89%. The formula is given as follows:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

Overall the Feed-Foward Neural Network and Random Forest Classifier models showed superior performance across all metrics (accuracy, precision, recall, and F1 Score) compared to the SVM. Both models achieved the same high accuracy and precision, with only a slight difference in recall and F1 Score. While still a strong performer, the SVM lagged behind the other two models in all evaluated metrics, indicating it may be less effective for this particular classification task.

Other Observations:

We originally included the 13 Haralick features in our feature extraction process, but despite their potential to capture texture information effectively, our experiments revealed that incorporating Haralick features led to an overall decrease in accuracy. As a result, we focused on other feature extraction techniques that yielded higher performance in our classification tasks.

5. Conclusion

The comparison of Feed-Forward Neural Network (FNN), Random Forest Classifier (RFC), and Support Vector Machine (SVM) reveals that both FNN and RFC outperform SVM when not using the 13 Haralick features. FNN's slightly higher F1 Score gives it an edge in effectiveness.

Our proposed approach involves using image processing and machine learning techniques with Support Vector Machine and other models to accurately identify specific banknote denominations. The results suggest that it can be used in real-world applications and other criteria brought up in Section 1.

The noteworthy performance observed in the image processing have been achieved through relevant preprocessing and enhancement techniques. Significant challenges included mitigating exaggerated image characteristics caused by brightness, noise, and rotation, as well as addressing the impact of a small dataset and the presence of various objects on each note. Attempts to enhance the images through techniques like histogram equalization may dull natural colors and lead to blurring. Additionally, the dataset used was too small, which ultimately produced higher evaluation metrics. The presence of various objects on each bill may have also impacted the outcomes. It is also possible that the processing techniques applied could have removed intricacies and other security features from the bank notes in the dataset. Moving forward, we recommend using a more controlled set of images, exploring alternative neural networks, and improving the bank note dataset.

References

- ¹C.-C. Chang, T.-X. Yu, and H.-Y. Yen, «Paper currency verification with support vector machines», IEEE Computer Society (2008).
- ²O. K. Oyedotun and A. Khashman, «Banknote recognition: investigating processing and cognition framework using competitive neural network», PubMed Central (2017).

- ³*Random forest algorithm in machine learning*, GeeksForGeeks, (2024) <https://www.geeksforgeeks.org/random-forest-algorithm-in-machine-learning/> (visited on 06/01/2024).
- ⁴N. A. J. Sufri, N. A. Rahmad, N. F. Ghazali, N. Shahar, and M. A. As'ari, «Vision based system for banknote recognition using different machine learning and deep learning approach», IEEE Control and System Graduate Research Colloquium (2019).
- ⁵*Support vector machine (svm) algorithm*, GeeksForGeeks, (2023) <https://www.geeksforgeeks.org/support-vector-machine-algorithm/> (visited on 05/31/2024).
- ⁶M. Terry-Jack, *Deep learning: feed forward neural networks (ffnns)*, Medium, (2019) <https://medium.com/@b.terryjack/introduction-to-deep-learning-feed-forward-neural-networks-ffnns-a-k-a-c688d83a309d> (visited on 06/02/2024).

Appendix A. Additional Information

Appendix A.1. Dataset Characteristics

fig. A.8 shows the data characteristics of the dataset.

Appendix A.2. Python Code: Feature Extraction

Feature extraction on the bank note images involves defining several functions for extracting the necessary features and then combine these features in fig. A.9. The `extract_features` function processes the input image through several steps (grayscale conversion, histogram equalization, noise reduction) and extracts key features (texture, shape, edges, and color) to aid in the classification of bank notes.

Below is a simple example of some code used to extract the various features from an image that can be used for bank note classification.

Appendix A.3. Feature Extraction on current South African R10 Bank Note.

```
Number of front images: 27
Number of back images: 27
10 values
Value distribution: 27
Description 1: 010_new
Description 2: 010_old
Description 3: 100_old
Description 4: 100_new
Description 5: 200_old
Description 6: 200_new
Description 7: 20_old
Description 8: 20_new
Description 9: 50_old
Description 10: 50_new
Description 11: 200_old
Description 12: 010_old
Description 13: 050_new
Description 14: 050_new
Description 15: 100_new
Description 16: 100_old
Description 17: 010_old
Description 18: 010_new
Description 19: 010_old_1
Description 20: 020_old
Description 21: 050_old
Description 22: 100_old_1
Description 23: 200_old
Description 24: R_010_old
Description 25: R_020_old
Description 26: R_050_old
Description 27: R_100_old
```

Figure A.8: Dataset information and characteristics.


```

def extract_features(image, pca_components=50):
    gray_image = get_gray_image(image)

    equalized_gray_image = equalize_image(gray_image)
    equalized_color_image = equalize_color_image(image)

    enhanced_color_image = noise_reduction_color(equalized_color_image)
    enhanced_gray_image = noise_reduction(gray_image)

    haralick = extract_haralick_features(enhanced_gray_image)
    hu_moments = extract_hu_moments(enhanced_gray_image)
    edge_features = extract_edge_features(enhanced_gray_image)
    color_moments = extract_color_moments(enhanced_color_image)

    return hu_moments + edge_features + color_moments

```

Figure A.9: Feature extraction of a bank note image.

Table A.5: Combined Feature Vector for the Current R10 Bank Note.

<i>FeatureValue</i>	<i>TypeofFeature</i>
0.0019	Hu Moment
$6.8650 \cdot 10^{-7}$	Hu Moment
$3.7154 \cdot 10^{-10}$	Hu Moment
$4.3999 \cdot 10^{-10}$	Hu Moment
$1.7380 \cdot 10^{-19}$	Hu Moment
$3.6389 \cdot 10^{-13}$	Hu Moment
$3.7987 \cdot 10^{-20}$	Hu Moment
15.2073	Edge Feature
92.9062	Colour Moment
71.8789	Colour Moment
0.2366	Colour Moment
144.1977	Colour Moment
70.7970	Colour Moment
-0.1271	Colour Moment
109.8422	Colour Moment
75.2179	Colour Moment
0.0792	Colour Moment